

Experiment 'p_6_2_i_has_adjacent_duplicates_array' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_6_2_i_has_adjacent_duplicates_array

Natural language statement: null

Method signature: p_6_2_i_has_adjacent_duplicates_array(inputs: array<int>) returns (has_duplicates: bool)

Ensures

- `has_duplicates == (exists i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1])`

Requires

- `inputs.Length >= 2`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_6_2_i_has_adjacent_duplicates_array(inputs: array<int>) returns (has_duplicates: bool)`

The method should respect the following contract:

requires `inputs.Length >= 2`, ensures `has_duplicates == (exists i :: 0 <= i < inputs.Length - 1 && inputs[i] == inputs[i + 1])`

Produce and show only the Dafny body of this method, including the curly braces that surround it. Do not show the signature nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnO8HYnCj5ztRxIWn9vtUmlDliq5

Model: gpt-5.5-2026-04-23

Created at: 1780663209

Finish reason: stop

Usage: 312 tokens in, and 5637 tokens out

```
{
  var i := 0;
  while i < inputs.Length - 1
    invariant 0 <= i <= inputs.Length - 1
    invariant forall j :: 0 <= j < i ==> inputs[j] != inputs[j + 1]
  {
    if inputs[i] == inputs[i + 1] {
      has_duplicates := true;
      assert 0 <= i < inputs.Length - 1;
      assert inputs[i] == inputs[i + 1];
      assert exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1];
      assert has_duplicates == (exists j :: 0 <= j < inputs.Length - 1 && inputs[j] ==
        ↪ inputs[j + 1]);
      return;
    }
    assert inputs[i] != inputs[i + 1];
    var oldI := i;
    assert forall j :: 0 <= j < oldI ==> inputs[j] != inputs[j + 1];
    i := i + 1;
    assert i == oldI + 1;
  }
}
```

```

    assert i <= inputs.Length - 1;
    forall j | 0 <= j < i
        ensures inputs[j] != inputs[j + 1]
    {
        if j == oldI {
            assert inputs[j] != inputs[j + 1];
        } else {
            assert j < oldI;
            assert inputs[j] != inputs[j + 1];
        }
    }
}
has_duplicates := false;
assert i == inputs.Length - 1;
assert forall j :: 0 <= j < inputs.Length - 1 ==> inputs[j] != inputs[j + 1];
assert !(exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]);
assert has_duplicates == (exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs
    ↪ [j + 1]);
}

```

Final Program

```

method p_6_2_i_has_adjacent_duplicates_array(inputs: array<int>) returns (has_duplicates:
    ↪ bool)
    requires inputs.Length >= 2
    ensures has_duplicates == (exists i :: 0 <= i < inputs.Length - 1 && inputs[i] ==
        ↪ inputs[i + 1])
{
    var i := 0;
    while i < inputs.Length - 1
        invariant 0 <= i <= inputs.Length - 1
        invariant forall j :: 0 <= j < i ==> inputs[j] != inputs[j + 1]
    {
        if inputs[i] == inputs[i + 1] {
            has_duplicates := true;
            assert 0 <= i < inputs.Length - 1;
            assert inputs[i] == inputs[i + 1];
            assert exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1];
            assert has_duplicates == (exists j :: 0 <= j < inputs.Length - 1 && inputs[j] ==
                ↪ inputs[j + 1]);
            return;
        }
        assert inputs[i] != inputs[i + 1];
        var oldI := i;
        assert forall j :: 0 <= j < oldI ==> inputs[j] != inputs[j + 1];
        i := i + 1;
        assert i == oldI + 1;
        assert i <= inputs.Length - 1;
        forall j | 0 <= j < i
            ensures inputs[j] != inputs[j + 1]
    }
}

```

```

{
  if j == oldI {
    assert inputs[j] != inputs[j + 1];
  } else {
    assert j < oldI;
    assert inputs[j] != inputs[j + 1];
  }
}
}
has_duplicates := false;
assert i == inputs.Length - 1;
assert forall j :: 0 <= j < inputs.Length - 1 ==> inputs[j] != inputs[j + 1];
assert !(exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs[j + 1]);
assert has_duplicates == (exists j :: 0 <= j < inputs.Length - 1 && inputs[j] == inputs
    ↪ [j + 1]);
}

```

Total Token Usage

Input tokens: 312

Output tokens: 5637

Reasoning tokens: 5120

Sum of 'total tokens': 5949

Experiment Timings

Overall Experiment started at 1780663208889, ended at 1780663336105, lasting 127216ms (127.22 seconds)

Iteration #1 started at 1780663208891, ended at 1780663336105, lasting 127214ms (127.21 seconds)